Performance Analysis of Convolutional Neural Network Models

Kala S*, Debdeep Paul†, Babita R Jose*, Jimson Mathew‡ and Nalesh S§
*Cochin University of Science And Technology, Kerala, India - 682 022
†Department of Electrical Engineering, IIT Patna, India
‡Department of Computer Science and Engineering, IIT Patna, India
Email:{debdeep.ee15, jimson}@iitp.ac.in
§Department of Electronics, Cochin University of Science And Technology, Kerala, India - 682 022
Email:{kalas, babitajose, nalesh}@cusat.ac.in

***Abstract*—Deep Neural Networks (DNNs) have become a promising solution for numerous machine learning tasks, which include object detection, classification, weather forecasting, video surveillance and safety management. Convolutional Neural Network (CNN) is a class of DNN, which can achieve state-of-art accuracy and high performance for most of the computer vision tasks. Training of CNNs is computationally intensive and may take multiple days for completion. In this work we have performed the feed forward path implementation and training of AlexNet, VGG-16 and ResNet models on Nvidia GeForce GTX 1080 Ti GPU with 3584 CUDA Cores, 1582 MHz, 11GB GDDR5X, using Caffe framework. Inference and training of these models are also performed on Nvidia Jetson TX2 hardware platform. We also present a detailed analysis of graphics processing unit (GPU) and general purpose processor (GPP) implementations of AlexNet, VGG-16 and ResNet-50 network models. A hardware architecture for performing convolution operations in feed forward path of CNN has also been implemented in Virtex-7 XC7VX690T FPGA, with throughput of 184 GFLOPS for AlexNet, 172 GFLOPS for VGG-16 and 197 GFLOPS for ResNet-50 model.**

***Keywords*-Convolutional neural networks, graphics processing unit, execution time, FPGA, training, performance.**

I. Introduction

Deep Neural Networks (DNNs) have emerged as a broad area in the field of artificial intelligence, which can outperform human accuracy in many applications. DNNs are explored for various applications like self driving cars, mobile robotics, cancer detection, and in entertainment field like computer gaming and so on. Convolutional Neural Network (CNN) is a state-of-art approach for solving various computer vision tasks. CNN has the ability to mimic human brain's behavior in solving many intelligent tasks. CNNs are widely used in machine learning applications such as image classification, object detection, speech processing and many more. Accuracy of CNN algorithm comes at the cost of its large computational complexity. CNNs are generally accelerated on either general purpose processors (GPP) or graphics processing unit (GPU). For real-time embedded applications, field programmable gate arrays (FPGA) are preferred owing to its reconfigurability and acceptable power dissipation.

978-1-7281-5523-4/19/$31.00

Several FPGA based implementations are available from various research groups like [2], [8], [1]. But they have used the pre-trained models for implementations on FPGA. Training of a deep network may take hours or even multiple days. Generally training of a deep neural network is performed on a high performance computing platform like GPU. Key contributions of this work are:

- We evaluate the execution time of various layers in the feed forward network (convolution, pooling, ReLU, fully connected) of popular CNN models like AlexNet, VGG-16 and ResNet on platforms like GPP, GPU, server and Jetson TX2 board.
- We evaluate the training period for these CNN models on same platforms using Caffe framework.
- We also present an architecture for performing convolution operations in the feed forward network of CNN. Hardware implementation of the architecture has been done in Virtex-7 FPGA platform, with high performance in terms of GFLOPS.

The organization of this paper is as follows. Background work and literature review is presented in Section II. Section III is a case study of most popular CNN models. Section IV describes the details of convolution architecture for CNNs. Section V gives the implementation and evaluation results. Section VI concludes the paper.

II. Background and Related works

CNNs are biologically inspired feed forward networks, with large computational complexity. CNN has an advantage of processing large amount of image data with less number of parameters compared to artificial neural networks (ANN). CNNs typically consist of different operations like convolutions, pooling, normalization and fully connected neural networks. Number of layers in CNNs vary from tens to hundreds.

Convolutional layers accept numerous input feature maps and convolution operation with kernel is performed on each of them, to get output. These outputs are added up to get the final feature output. A non linear activation function (for example, rectified linear unit (ReLU), sigmoid function) is applied to each of these output neurons. ReLU is commonly

used for enabling faster training and various types of ReLU are also used in-order to get maximum precision. In pooling layer, which follows the convolution layer, sub sampling is performed, where the size of feature map (dimensionality) is reduced in each channel. Most commonly used pooling techniques are max-pooling and average pooling. In fully connected layer, each neuron is connected with every other neuron, and thus number of weights are very large in this layer. Hence fully connected layers are memory intensive compared to other layers.

Convolution layers account for majority of the computations in a CNN whereas, fully connected layers are memory bound. AlexNet, which was the first CNN model to win the ImageNet challenge in 2012, has five CONV layers and three FC layers. Overfeat also has five CONV and three FC layers, but with more number of kernels. VGG has three models, VGG-11, VGG-16 and VGG-19, which has deeper layers. GoogleNet goes much more deeper, with 22 layers. ResNet has exceeded human level accuracy, with more than 34 layers.

Several GPU accelerators for CNN are available in literature. In [3], performance evaluation of CNN on Tesla K40c, for various kind of implementations is presented. Training period of AlexNet models are evaluated for various frameworks using GPU in [4]. FFT based CNN implementations using Torch 7 environment were performed on GeForce GTX Titan GPU in [5]. GPU accelerators for CNN is also presented in [6]. Many open-source frameworks for deep learning like Caffe, Torch, Theano etc. were also introduced which can use CUDA programming. Specific libraries like cuDNN are also available for accelerating CNNs.

FPGA implementations of CNNs are presented in [8], [7]. In [8], both Winograd minimal filtering based and conventional convolution architecture is presented for FPGA implementation of AlexNet model. Open CL based FPGA accelerator for CNN is presented in [7], which implements both AlexNet and VGG models. Implementation of ResNet model on FPGA is not explored widely by researchers. Some research papers like [9] has implemented various ResNet models on FPGA.

In this paper we present inference and training of popular CNN models on various platforms like CPU, GPU, server, Jetson TX2 and also an efficient convolution architecture for inference on FPGA platform.

III. CNN Models

For evaluation purpose, we have considered three popular CNN models which are discussed in III-A, III-B and III-C.

A. AlexNet Model

AlexNet was the first CNN model to win the ImageNet challenge in 2012. AlexNet CNN model is used for image classification tasks which can classify thousand categories. AlexNet consists of five convolution (CONV) layers and three FC layers. Kernels used in AlexNet are of sizes 11, 5 and 3. Configurations of AlexNet model are given in Table I. First layer consists of three channels, which correspond to RGB components of the input feature map. First layer uses a stride of 4 while rest of the layers use a stride of one. In ImageNet Challenge 2012, the 96 output channels of first CONV layer of AlexNet were split into two 48 channels to reduce the number of weights and thereby minimizing the computational complexity. Number of multiply and accumulate (MAC) operations in convolutional and fully connected layers are shown in Fig. 1.

TABLE I
AlexNet Layer Configuration

Layers	$Feature_{in}$	Kernel	#Kernels	#Channels	Stride
CONV1	227×227	11×11	96	3	4
POOL1	55×55	3×3		96	2
NORM1	27×27			96	
CONV2	27×27	5×5	256	96	1
POOL2	27×27	3×3		256	2
NORM2	13×13		256	96	
CONV3	13×13	3×3	384	256	1
CONV4	13×13	3×3	384	192	1
CONV5	13×13	3×3	256	192	1
POOL3	13×13	3×3		256	2
FC6	4096 neurons				
FC7	4096 neurons				
FC8	1000 neurons				

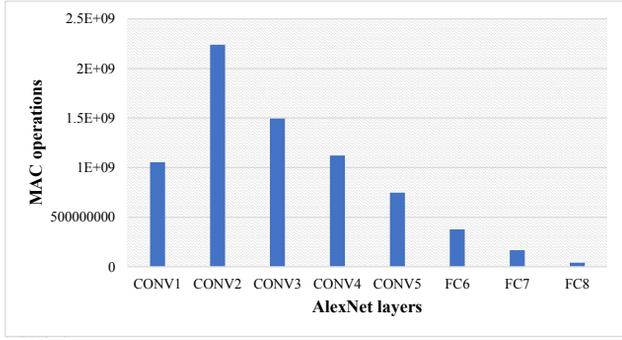

Fig. 1. MAC Operations in AlexNet

B. VGG-16 Model

VGG model got second place in ILSVRC challenge in 2014, with top-5 accuracy of 92.6%. VGG network model is available in three different versions, namely VGG-11, VGG-16 and VGG-19, based on the number of layers. Number of convolution stages in VGG network model is given is Table II. For example, VGG-11 has a single CONV layer in the first and second stage, all other CONV stages have two layers and there are three FC layers, with a total of 11 layers. VGG-16 network model consists of five stages of convolution layers and three fully connected layers, and operations like max pooling and soft max. Number of MAC operations in convolutional and fully connected layers are shown in Fig. 2. As seen in Fig. 1 and Fig. 2, computations are high in convolutional layers than in fully connected layers.

TABLE II
VGG NETWORK MODELS

VGG Model	CONV stage 1	CONV stage 2	CONV stage 3	CONV stage 4	CONV stage 5	FC	Total
VGG-11	1	1	2	2	2	3	11
VGG-16	2	2	3	3	3	3	16
VGG-19	2	2	4	4	4	4	19

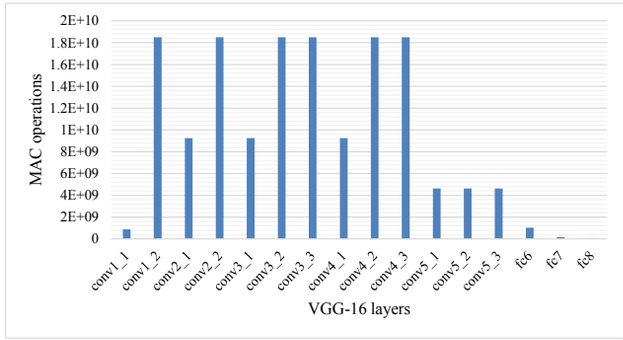

Fig. 2. MAC Operations in VGG-16

C. ResNet Model

ResNet model won ISLVRC challenge in 2015 with top-5 error of 3.57%. ResNet uses residual connections to avoid performance degradation and form highly deep networks. For ImageNet challenge, a depth of 152 layers were used in ResNet which has only one fully connected layer as the last layer. Each of the residual block has two 3×3 convolutional layers. ResNet uses depth of 34, 50, 101 or 152 layers for realization.

IV. Proposed Convolution Architecture

In this section we discuss an efficient hardware architecture for inference of CNN models. Convolutional layers account for 90% of the computations in CNN. Hence we restrict our focus on CONV layers only in this work. We present a suitable architecture for performing convolution operations in the feed forward path of CNN. Convolution involves multiply and accumulate (MAC) operations and it use matrix multipliers. In Fig. 3 we give a generic architecture for performing convolutions in CNN. The architecture uses a series of MAC units in parallel for improving the throughput. Selection of parallelism depends on the available hardware and bandwidth.

Proposed architecture performs convolution of an $M \times M$ input feature map with a $K \times K$ kernel, with C input channels and F number of kernels. Input features and kernel coefficients are accessed from DDR and the output feature is also written back to DDR. Single channel of M×M input feature is considered first, which has to be convolved with F kernels of size K×K each. Initially f data are fed in serial to the processing engine. We use x MAC units for performing

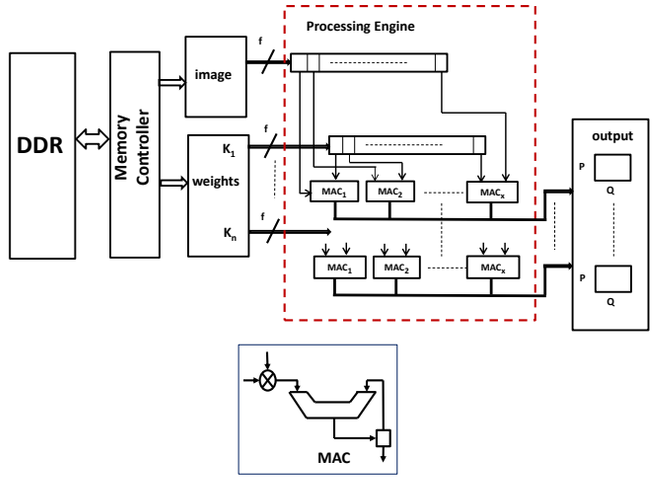

Fig. 3. Proposed Convolution Architecture

a row convolution ($x = f$). We have n such row convolution units which perform convolution operation in parallel, denoted as K_1 to K_n. Outputs of all the channels are added together, reusing the accumulators in MAC unit resulting in n number of $P \times Q$ outputs.

In FPGA implementation of proposed architecture, we have used 11 MAC units in each row, (ie, $x = 11$) and 64 channels are processed in parallel ie., $n = 64$. Our architecture supports kernel sizes, 11×11, 7×7, 5×5, 3×3 and 1×1. This architecture can be used to accelerate popular CNN models like AlexNet, VGG-16 and ResNet. We perform batch processing to accommodate all channels of these CNN models. 64 channels were taken in parallel since VGG-16 and ResNet were having 64 channels, which is the minimum number of channels and a maximum number of 384 is being used in AlexNet as seen from Table I.

V. Implementation and Results

We have performed the analysis in four different platforms. GPU simulations are performed on Nvidia GeForce GTX 1080 Ti GPU with 3584 CUDA Cores, 1582 MHz, 11GB GDDR5X. We have used CUDA Version 8.0.61 and CuDNN library with Version 6.0.21. General purpose processor platform used for the simulation is Intel Core i7 7^{th} generation CPU, with frequency 3.60 GHz and 8GB RAM. Server simulations were done on Intel Xeon E5 @ 2.20 GHz server. For inference, no other workloads were distributed over CPU and GPU. We have also performed the implementation of AlexNet, VGG-16 and ResNet-50 on Nvidia Jetson TX2 platform. Jetson TX2 is a fast and power-efficient embedded device for applications involving deep learning techniques. We have used ImageNet dataset for performance evaluation, with input size 224×224.

Feed forward network simulations of AlexNet model were performed on CPU and server, which is shown in Fig.4. Fig. 4 shows only convolution layers, for better clarity. GPU simulations for feed forward network layers of AlexNet model are given in Fig. 5. Fig. 6 shows the execution time analysis of

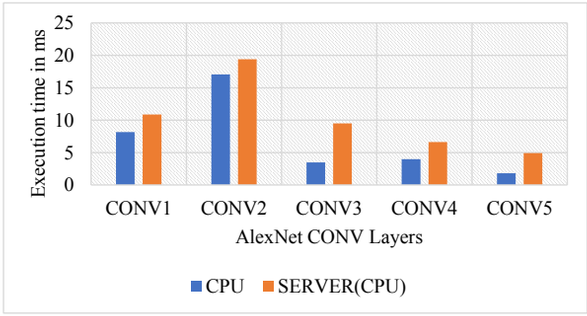

Fig. 4. AlexNet feed forward network simulations

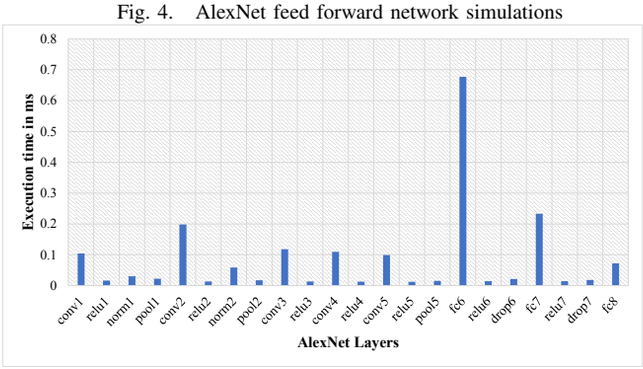

Fig. 5. AlexNet feed forward network simulations on GPU

all layers of AlexNet during feed forward path, in all the three platforms. Similar experiments have been performed for VGG-16 and ResNet-50 models also. Fig. 7 shows the execution time for feed forward path of AlexNet, VGG-16 and ResNet models in CPU, GPU and Nvidia Jetson TX2 platforms. Execution time for inference is much faster in GPU compared to general purpose processor and Jetson TX2 platform. Fig. 8 shows the training time for AlexNet, VGG-16 and ResNet-50 models in CPU, GPU and Nvidia Jetson TX2 platforms. As seen from Fig. 8, training period is much shorter in GPU compared to CPU and Jetson hardware simulations.

A. *FPGA Implementation*

Hardware implementation of convolution architecture has been performed on Virtex-7 XC7VX690T FPGA platform. RTL has been done using Verilog HDL and the design has been synthesized and implemented with Xilinx Vivado 2016.4. We have used 32-bits floating point arithmetic for implementation.

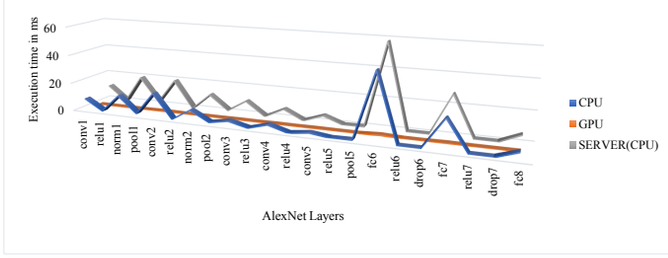

Fig. 6. Comparison of AlexNet feed forward simulations

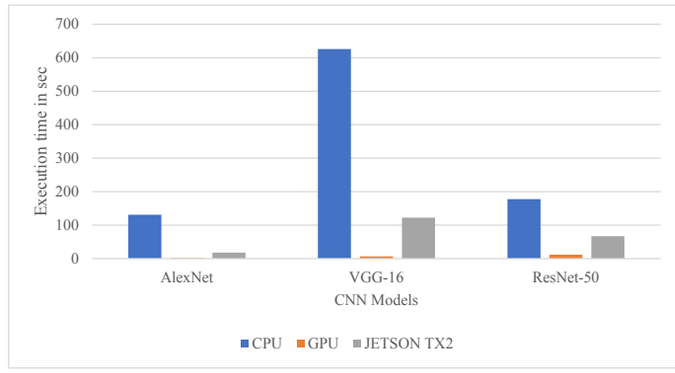

Fig. 7. Comparison of feed forward simulations

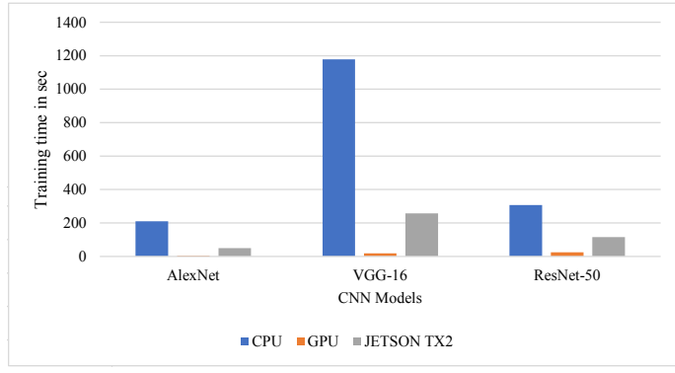

Fig. 8. Comparison of training periods

The architecture has been verified using Matlab simulation environment. Proposed architecture has an operating frequency of 150 MHz. Resource Utilization of the architecture is given in Table III.

Convolution layers of AlexNet, VGG-16 and ResNet-50 were implemented on FPGA using proposed architecture and the performance was evaluated. Performance of each of the convolution layers of AlexNet is given in Table IV. Table V gives the throughput in terms of giga floating point operations per second (GFLOPS) for different CNN models.

TABLE III
RESOURCE UTILIZATION

Resource	Used	Available	Percentage utilization
Slice LUTs	348608	1221600	28.53
DSP48E	1408	2160	65.18
Flip Flops	91072	2443200	3.7

TABLE IV
PERFORMANCE OF DIFFERENT CONV LAYERS OF ALEXNET

CONV Layer	Performance (GFLOPS)
Layer 1	211.2
Layer 2	192
Layer 3	172.8
Layer 4	172.8
Layer 5	172.8

TABLE V
PERFORMANCE OF DIFFERENT CNN MODELS

Network	# CONV layers	Performance (GFLOPS)
AlexNet	5	184.32
VGG-16	13	172.8
ResNet-50	49	197.09

VI. CONCLUSION

CNNs are state-of-art technique for many machine learning applications and it involves large number of computations and memory. Inference and training of CNN models will take large amount of time which affects the performance of the network. In this work we have done the performance analysis of various popular CNN models like AlexNet, VGG-16 and ResNet-50 on different platforms like CPU, GPU, server and Nvidia Jetson TX2 hardware. We report the inference time and training time of these CNN models and compare them on these platforms. We have also proposed a generic convolution architecture for realizing the convolution layers of CNN and FPGA implementation has been performed on Virtex-7. The performance of our architecture is 184.32 GFLOPS for AlexNet, 172.8 GFLOPS for VGG-16 and 197 GFLOPS for ResNet-50.

ACKNOWLEDGMENT

This work was supported in part by Kerala State Council for Science, Technology and Environment (KSCSTE), Kerala, India, through Woman Scientist Division (WSD-BLP) project grant order No 742/2018/KSCSTE.

REFERENCES

[1] J. Wang et al., "Efficient Hardware Architectures for Deep Convolutional Neural Network," in IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 65, no. 6, pp. 1941-1953, June 2018. doi: 10.1109/TCSI.2017.2767204

[2] Q. Xiao et al., "Exploring heterogeneous algorithms for accelerating deep convolutional neural networks on FPGAs," 2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC), Austin, TX, 2017, pp. 1-6. doi: 10.1145/3061639.3062244

[3] X. Li et al., "Performance Analysis of GPU-Based Convolutional Neural Networks," 2016 45th Int'l Conference on Parallel Processing (ICPP), Philadelphia, PA, 2016, pp. 67-76. doi: 10.1109/ICPP.2016.15

[4] H. Kim et al., "Performance analysis of CNN frameworks for GPUs," 2017 IEEE ISPASS, Santa Rosa, CA, 2017, pp. 55-64. doi: 10.1109/ISPASS.2017.7975270

[5] Michael Mathieu, Mikael Henaff, and Yann LeCun. 2014. Fast Training of Convolutional Networks through FFTs. In Proceedings of ICLR 2014. arXiv:1312.5851

[6] Srimat Chakradhar et al., A dynamically configurable coprocessor for convolutional neural networks. SIGARCH Comput. Archit. News 38, 3 (June 2010), 247-257. DOI: https://doi.org/10.1145/1816038.1815993

[7] Naveen Suda et al., 2016. Throughput-Optimized OpenCL-based FPGA Accelerator for Large-Scale Convolutional Neural Networks. In Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '16). ACM, New York, NY, USA, 16-25. DOI: https://doi.org/10.1145/2847263.2847276

[8] K. S, J. Mathew, B. R. Jose and N. S, "UniWiG: Unified Winograd-GEMM Architecture for Accelerating CNN on FPGAs," 2019 32nd International Conference on VLSI Design and 2019 18th International Conference on Embedded Systems (VLSID), Delhi, NCR, India, 2019, pp. 209-214. doi: 10.1109/VLSID.2019.00055

[9] Y. Ma, Y. Cao, S. Vrudhula and J. Seo, "Optimizing the Convolution Operation to Accelerate Deep Neural Networks on FPGA," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 26, no. 7, pp. 1354-1367, July 2018. doi: 10.1109/TVLSI.2018.2815603